

On This Page >



05 Deployment Parameters

Status

accepted

Context

Currently different operations (install, update) have different modes. Install always evals locally and pushes the derivation to a remote system. update has a configurable buildHost and targetHost. Confusingly install always evals locally and update always evals on the targetHost, so hosts have different semantics in different operations contexts.

Decision

Add evalHost to make this clear and configurable for the user. This would leave us with:

- evalHost
- buildHost
- targetHost

for the update and install operation.

`evalHost` would be the machine that evaluates the nixos configuration. if evalHost is not localhost, we upload the non secret vars and the nix archived flake (this is usually the same operation) to the evalMachine.

`buildHost` would be what is used by the machine to build, it would correspond to `--build-host` on the nixos-rebuild command or `--builders` for nix build.

`targetHost` would be the machine where the closure gets copied to and activated (either through install or switch-to-configuration). It corresponds to `--targetHost` for `nixos-rebuild` or where we usually point `nixos-anywhere` to.

This hosts could be set either through CLI args (or forms for the GUI) or via the inventory. If both are given, the CLI args would take precedence.

Consequences

We now support every deployment model of every tool out there with a bunch of simple flags. The semantics are more clear and we can write some nice documentation.

The install code has to be reworked, since `nixos-anywhere` has problems with `evalHost` and `targetHost` being the same machine, So we would need to `kexec` first and use the `kexec` image (or installer) as the `evalHost` afterwards.

In cases where the `evalHost` doesn't have access to the `targetHost` or `buildHost`, we need to setup temporary entries for the lifetime of the command.

← Previous

04 Fetching Nix From Python

Next →
Title

Need help? Reach out to [our community](#).